

Mini-SoC Integration Lab on GITHUB

Introduction

This tutorial guides step-by-step through building a small, modular mini-SoC composed of three simple IP blocks (a 4-bit counter, a 2-bit ALU, and a 2x1 MUX), integrating them in a top module, simulating with Verilator, and visualizing waveforms using GTKWave. The project is designed to be hosted on GitHub and run locally or in CI.

Objective

- Create an easily readable GitHub project implementing the mini-SoC.
- Simulate the SoC using Verilator and capture waveforms.
- Visualize signals with GTKWave.

Step 1: Create the GitHub repo (locally or on GitHub)

Option A : Create locally and push:

```
mkdir mini_soc  
cd mini_soc  
git init
```

Option B : Create repo on GitHub website then git clone it, or use GitHub CLI:

```
gh repo create mini_soc --public --confirm  
# then clone:  
git clone https://github.com/<yourusername>/mini_soc.git  
cd mini_soc
```

Step 2: Add source files

Create the following files and write the codes.

a) 4-bit synchronous up_counter (counter.v)

```
// 4-bit synchronous up counter with synchronous reset  
module counter (  
    input wire clk,  
  
    input wire rst,  
    output reg [3:0] count  
);  
  
    always @(posedge clk) begin  
        if (rst)
```

```
count <= 4'd0;

    else

        count <=
count + 1;    end
endmodule
```

b) Simple 2-bit ALU (add / subtract) (alu.v)

// 2-bit ALU: sel=0 (add) , sel=1(sub)

```
module alu (    input  wire [1:0] a,
input  wire [1:0] b,
input  wire      sel,
output reg  [1:0] result
);
    always @(*) begin
        case (sel)
            1'b0: result = a + b;
            1'b1: result = a - b;
            default: result = 2'b00;
        endcase
    end
endmodule
```

2) 2-bit multiplexer (mux.v)

```
// 2-bit 2:1 multiplexer
module mux2x1 (
input  wire [1:0] in0,
input  wire [1:0] in1,
input  wire      sel,
output wire [1:0] out );
    assign out = sel ? in1 : in0;
endmodule
```

d) SoC Top (mini_soc.v)

// Integrates counter, ALU, and MUX

```
module mini_soc (  
    input clk,  
    input rst,  
    input alu_sel,  
    input mux_sel,  
    output [1:0] soc_out  
);  
    wire [3:0] count;  
    wire [1:0] alu_out;  
    // Instantiate IP blocks  
    counter u_counter (.clk(clk), .rst(rst), .count(count));  
    alu u_alu (.a(count[3:2]), .b(count[1:0]), .sel(alu_sel),  
    .result(alu_out));  
    mux2x1 u_mux (.in0(count[3:2]), .in1(alu_out),  
    .sel(mux_sel), .out(soc_out));  
endmodule
```

The testbench for SoC: mini_soc_tb.v

```
module mini_soc_tb;  
    reg clk = 0;  
    reg rst = 1;  
    reg alu_sel = 0;  
    reg mux_sel = 0;  
    wire [1:0] soc_out;  
    always #5 clk = ~clk;  
    mini_soc dut (  
        .clk(clk),  
        .rst(rst),  
        .alu_sel(alu_sel),  
        .mux_sel(mux_sel),
```

```
.soc_out(soc_out)

);

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, mini_soc_tb);
    // Reset release
    #10 rst = 0;
    // Stimulate ALU and MUX selections
    #20 alu_sel = 1;
    #30 mux_sel = 1;
    #40 alu_sel = 0;
    #20 mux_sel = 0;
    #50 $finish;
end
endmodule
```

Step 3 : Script file

Create script file run.sh and make it executable using command “chmod +x run.sh”. This script builds and runs the Verilator simulation and opens GTKWave.

```
#!/bin/bash

# Use Verilator to compile and generate simulation files
verilator --binary -j 0 -Wall \
    counter.v alu.v mux2x1.v mini_soc.v mini_soc_tb.v \
    --top mini_soc_tb --timing --CFLAGS "-std=c++20" --trace
# Step 2: Enter the object directory
cd obj_dir || { echo "obj_dir not found"; exit 1; }
# Step 3: Compile simulation binary
make -f Vmini_soc_tb.mk Vmini_soc_tb || { echo "Compilation
failed"; exit 1; }
# Step 4: Run the simulation
./Vmini_soc_tb || { echo "Simulation failed"; exit 1; }
# Step 5: Launch GTKWave to view the waveform
gtkwave dump.vcd
```

Step 4: Commit & push to GitHub

```
git add .  
  
git commit -m "Add Mini-SoC lab: counter, alu, mux, testbench,  
run script"  
  
git branch -M main  
  
git remote add origin \  
https://github.com/<yourusername>/mini_soc.git # if not set  
  
git push -u origin main  
  
(Replace <your-username> with your GitHub username or use gh  
repo create as described earlier.)
```

Step 5: Run locally

From repository root:

```
./run.sh
```

- The simulation will produce dump.vcd in the project root.
- GTKWave will open and let you inspect signals such as clk, rst, count, alu_out, soc_out.

GTKWave Waveform:

- **Counter Operation:** count increments on each clock cycle after reset is released.
- **ALU Behavior:** alu_out switches between addition and subtraction depending on the value of alu_sel.
- **MUX Output:** soc_out toggles between the counter's MSBs and the ALU output based on mux_sel.
- Control signal changes are reflected promptly in the waveform, enabling easy tracing of data flow and IP operation.

Conclusion

You now have a GitHub ready project that demonstrates basic IP integration in a mini-SoC, with a full simulation flow (Verilator & GTKWave) and an optional CI workflow for automated runs. This lab helps beginners grasp modular RTL design, simple data-path integration, and how to set up a reproducible simulation workflow on GitHub.